Meet the OS You Will Build

Lab 2 · Track B: build your own small OS (xv6) · 90 minutes

Goal

Over this semester you will not only study operating systems, you will grow one. Our teaching OS is xv6: a small, real, bootable Unix (a modern re-implementation of Unix Version 6, in about 9,000 lines of C, running on RISC-V under the QEMU emulator). Today you build it from source, boot it, use it as the tiny Unix it is, read some of its code, and write your very first program that runs inside it.

Before you start

- Work on the provided Linux VM. The cross-compiler (riscv64) and QEMU are already installed.
- The course copy of xv6 is at ~/xv6-riscv (or clone it as shown below).

```
cd ~/xv6-riscv 2>/dev/null || \
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls                      # kernel/  user/  Makefile  README ...
```

If a tool is missing (assistant will have handled this)

On a fresh image the toolchain is: sudo apt update && sudo apt install -y git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu. On the course VM this is already baked in.

Part 1. Build it and boot it (~15 min)

1. Compile the kernel and the user programs, then launch xv6 in the emulator:

```
make qemu
```

After a few seconds you should see the kernel come up and drop you into a shell:

```
xv6 kernel is booting

init: starting sh
$
```

How to leave xv6

xv6 runs inside QEMU. To quit the emulator, press Ctrl-A, release, then press X. (That is the QEMU exit sequence, not a shortcut inside xv6.)

Part 2. Use xv6 as the Unix it is (~20 min)

At the xv6 $ prompt, run its built-in programs. Notice how familiar they feel: this is the ancestor of the Linux you used in Lab 1.

```
$ ls
$ cat README
$ echo hello xv6
$ ls | grep c        # pipes work here too
$ wc README
$ usertests -q       # optional: the kernel test suite (takes a while)
```

Observe (write in notes.md)

(1) List three programs xv6 ships with. (2) You just used a pipe (|) inside xv6. Which two OS features must exist for a pipe to work? (Hint: think processes and communication.) (3) In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?

Part 3. Read the source (~20 min)

Quit QEMU (Ctrl-A then X). Open the tree in your editor. The whole OS splits into two folders, and this split is the heart of how an OS is organised.

- kernel/ is the operating system itself (runs privileged): scheduler, memory, file system, system-call handlers.
- user/ holds the ordinary programs that run on top of it: ls, cat, echo, the shell.

2. Read a user program (it is tiny and you will recognise the shape from your C labs):

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
```

3. Notice the system calls. In user/cat.c, read() and write() are not ordinary functions. They are requests into kernel/. Find where they are declared:

```
cat user/user.h        # the list of system calls available to programs
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```

Observe (write in notes.md)

(1) Which system calls does user/cat.c use, and what does each one ask the kernel to do? (2) In which kernel file, and on which line, is sys_read implemented? (3) In one sentence: what is the difference between the code in kernel/ and the code in user/?

The big idea

On Linux (Lab 1) you could only see programs from the outside. In xv6 you can read BOTH sides: the program that asks (user/) and the kernel code that answers (kernel/). That is why we build on xv6: it is small enough to hold the whole OS in your head.

Part 4. Write your first xv6 program (~30 min)

You will add a new user program, sleep, that pauses for a number of clock ticks by calling the sleep system call (which already exists in the kernel). This is the classic first exercise on xv6.

4. Create user/sleep.c:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

5. Tell the build about your new program. Open the Makefile, find the UPROGS list, and add the line for sleep. Copy the indentation of the line above it exactly (it is a tab, not spaces) and keep the backslash at the end:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_sleep\        # <-- add this line
    ...
```

6. Rebuild, boot, and test your program inside xv6:

```
make qemu
$ sleep 10          # should pause, then return to the prompt
$ sleep             # should print your usage message
```

What you just did

You wrote a program that makes a real system call into a kernel you compiled yourself, and ran it on an OS booting on emulated hardware. Next week we start changing the kernel side too.

Wrap-up and deliverable (~5 min)

You submit:

- user/sleep.c: your working program.
- notes.md: your short answers to the two "Observe" boxes (Part 2 and Part 3).

Push both to your course repository. The assistant will build your xv6 and run `sleep 10` to check it.

Grading (out of 10):

Criterion	Points
xv6 builds and boots; you used it and answered Part 2	3
Source-reading answers (kernel/ vs user/, system calls)	3
sleep.c works: correct usage check and calls sleep()	4

Finished early? (optional, not graded)

Write `user/pingpong.c`: the program creates a child with fork() and two pipes. The parent sends one byte to the child, the child prints "<pid>: received ping" and sends the byte back, then the parent prints "<pid>: received pong". Add it to UPROGS the same way as sleep.